

Misorientations

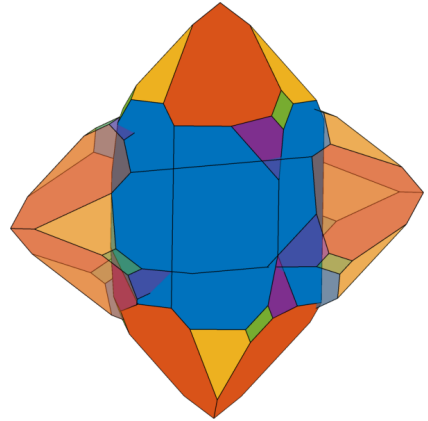
Ralf Hielscher

Faculty of Mathematics and Computer Science,
TU Bergakademie Freiberg, Germany

MTEX Workshop 2024

Misorientations

- ▶ describe the relative alignment of two crystal lattices with respect to each other
- ▶ if the lattices are of same phase it is the rotation that rotates one lattice onto the other.
- ▶ if the lattices of different phase it is the rotation between the corresponding orthonormal reference frames
- ▶ is the coordinate transform from lattice A coordinates into lattice B coordinates
- ▶ is subject to the symmetries of lattice A as well as lattice B

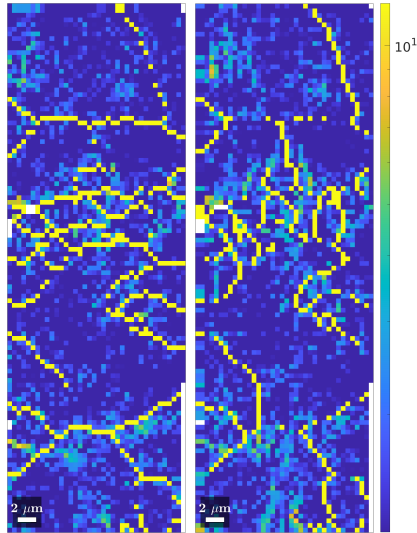


```
plot (cS)  
hold on  
plot (mori * cS)
```

Examples

neighboring pixels

```
ebsd.gradientX, ebsd.KAM
```



```
ori = ebsd.orientation;  
angle(ori(:,1:end-1),ori(:,2:end))
```

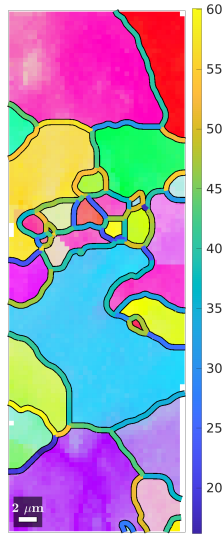
Examples

neighboring pixels

```
ebsd.gradientX , ebsd.KAM
```

grain boundary misorientations

```
grains.boundary.misorientation
```



```
gB = grains.boundary  
mori = gB.misorientation  
plot(gB, mori.angle ./ degree)
```


Examples

neighboring pixels

`ebsd.gradientX` , `ebsd.KAM`

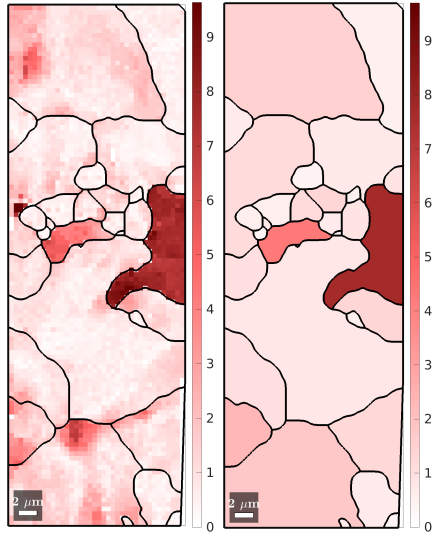
grain boundary misorientations

`grains.boundary.misorientation`

pixel to grain mean orientation

`ebsd.calcGROD(grains)`

`grains.GOS`



```
plot(ebsd.mis2mean.angle./ degree)  
plot(grains.GOS)
```

Examples

neighboring pixels

```
ebsd.gradientX, ebsd.KAM
```

grain boundary misorientations

```
grains.boundary.misorientation
```

pixel to grain mean orientation

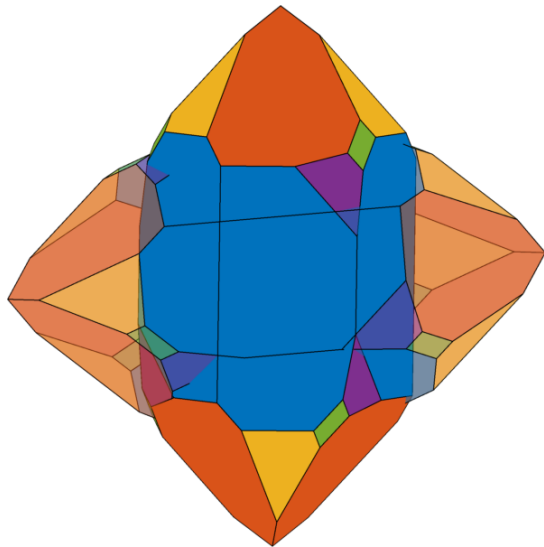
```
ebsd.calcGROD(grains)
```

```
grains.GOS
```

twinning misorientations

```
xi = Miller(1,0,-1,1,CS)
```

```
orientation.byAxisAngle(xi, pi)
```



Examples

neighboring pixels

```
ebsd.gradientX, ebsd.KAM
```

grain boundary misorientations

```
grains.boundary.misorientation
```

pixel to grain mean orientation

```
ebsd.calcGROD(grains)
```

```
grains.GOS
```

twinning misorientations

```
xi = Miller(1,0,-1,1,CS)  
orientation.byAxisAngle(xi, pi)
```

common site lattice misorientations

```
mori = CSL(3, cs)
```



Examples

neighboring pixels

```
ebsd.gradientX , ebsd.KAM
```

grain boundary misorientations

```
grains.boundary.misorientation
```

pixel to grain mean orientation

```
ebsd.calcGROD( grains )
```

```
grains.GOS
```

twinning misorientations

```
xi = Miller(1,0,-1,1,CS)
```

```
orientation.byAxisAngle( xi , pi )
```

common site lattice misorientations

```
mori = CSL(3,cs)
```

parent to child orientation relationship

```
orientation.burger( csB , csA )
```

Coordinate Transforms

Remember, an orientation converts crystal into specimen coordinates

```
oBeta = orientation.byEuler(0,0,0,csBeta)
r = oBeta * Miller(1,1,0,csBeta)
```

```
oBeta = orientation (Ti \(BETA\) -> xyz)

Bunge Euler angles in degree
phi1  Phi phi2 Inv.
    0    0    0    0

r = vector3d
      x      y      z
0.302024 0.302024 0
```

The inverse of an orientation converts specimen into crystal coordinates

```
oAlpha = orientation.byEuler(135, 90, 55, csAlpha)
inv(oAlpha) * r
```

Hence, **inv(aAlpha) * oBeta** converts beta coordinates into alpha coordinates

```
inv(oAlpha) * oBeta * Miller(1,1,0,csBeta)
```

Coordinate Transforms

Remember, an orientation converts crystal into specimen coordinates

```
oBeta = orientation.byEuler(0,0,0,csBeta)
r = oBeta * Miller(1,1,0,csBeta)
```

The inverse of an orientation converts specimen into crystal coordinates

```
oAlpha = orientation.byEuler(135, 90, 55, csAlpha)
inv(oAlpha) * r
```

```
oAlpha = orientation (Ti (alpha) -> xyz)
```

```
Bunge Euler angles in degree
```

```
phi1  Phi phi2 Inv.
```

```
135   90   55   0
```

```
ans = Miller (Ti (alpha))
```

```
h      k      i      l
```

```
0      0      0 1.9994
```

Hence, **inv(aAlpha) * oBeta** converts beta coordinates into alpha coordinates

```
inv(oAlpha) * oBeta * Miller(1,1,0,csBeta)
```

Coordinate Transforms

Remember, an orientation converts crystal into specimen coordinates

```
oBeta = orientation.byEuler(0,0,0,csBeta)
r = oBeta * Miller(1,1,0,csBeta)
```

The inverse of an orientation converts specimen into crystal coordinates

```
oAlpha = orientation.byEuler(135, 90, 55, csAlpha)
inv(oAlpha) * r
```

Hence, **inv(aAlpha) * oBeta** converts beta coordinates into alpha coordinates

```
inv(oAlpha) * oBeta * Miller(1,1,0,csBeta)
```

```
ans = Miller (Ti (alpha))
```

h	k	i	l
0	0	0	1.9994

Misorientations

The **misorientation beta2alpha** transforms Beta coordinates into Alpha coordinates

```
beta2alpha = inv(oAlpha) * oBeta  
beta2alpha * Miller(1,1,0,csBeta)
```

```
beta2alpha = misorientation (Ti (beta) -> Ti (alpha))
```

```
Bunge Euler angles in degree
```

```
phi1  Phi phi2 Inv.
```

```
125   90   45    0
```

```
ans = Miller
```

```
mineral: Ti (alpha) (622, X||a*, Y||b, Z||c*)
```

```
h      k      i      l
```

```
0      0      0 1.9994
```

its inverse transforms Alpha coordinates into Beta coordinates

```
alpha2beta = inv(beta2alpha)  
alpha2beta * Miller(0,0,0,1,csAlpha)
```

beta2alpha is close to the Burger orientation relationship between the hexagonal alpha and the cubic beta phase.

```
round2Miller(beta2alpha)
```


Misorientations

The **misorientation beta2alpha** transforms Beta coordinates into Alpha coordinates

```
beta2alpha = inv(oAlpha) * oBeta  
beta2alpha * Miller(1,1,0,csBeta)
```

its inverse transforms Alpha coordinates into Beta coordinates

```
alpha2beta = inv(beta2alpha)  
alpha2beta * Miller(0,0,0,1,csAlpha)
```

```
alpha2beta = misorientation (Ti (alpha) -> Ti (beta))
```

Bunge Euler angles in degree

phi1	Phi	phi2	Inv.
125	90	45	0

```
ans = Miller (Ti (beta))
```

h	k	l
0.5002	0.5002	0

beta2alpha is close to the Burger orientation relationship between the hexagonal alpha and the cubic beta phase.

```
round2Miller(beta2alpha)
```

Misorientations

The **misorientation beta2alpha** transforms Beta coordinates into Alpha coordinates

```
beta2alpha = inv(oAlpha) * oBeta  
beta2alpha * Miller(1,1,0,csBeta)
```

its inverse transforms Alpha coordinates into Beta coordinates

```
alpha2beta = inv(beta2alpha)  
alpha2beta * Miller(0,0,0,1,csAlpha)
```

beta2alpha is close to the Burger orientation relationship between the hexagonal alpha and the cubic beta phase.

```
round2Miller(beta2alpha)
```

```
(110) || (0001) [1-11] || [-12-10] error: 0.26
```

define the Burger orientation relationship exactly

```
beta2alpha = orientation.map(...  
    Miller(1,1,0,csBeta, 'hkl'), Miller(0,0,0,1,csAlpha, 'hkl'), ...  
    Miller(-1,1,-1,csBeta, 'uvw'), Miller(2,-1,-1,0,csAlpha, 'uvw'))
```

Misorientations

The **misorientation beta2alpha** transforms Beta coordinates into Alpha coordinates

```
beta2alpha = inv(oAlpha) * oBeta  
beta2alpha * Miller(1,1,0,csBeta)
```

its inverse transforms Alpha coordinates into Beta coordinates

```
alpha2beta = inv(beta2alpha)  
alpha2beta * Miller(0,0,0,1,csAlpha)
```

beta2alpha is close to the Burger orientation relationship between the hexagonal alpha and the cubic beta phase.

```
round2Miller(beta2alpha)
```

```
(110) || (0001) [1-11] || [-12-10] error: 0.26
```

define the Burger orientation relationship exactly

```
beta2alpha = orientation.map(...  
    Miller(1,1,0,csBeta, 'hkl'), Miller(0,0,0,1,csAlpha, 'hkl'), ...  
    Miller(-1,1,-1,csBeta, 'uvw'), Miller(2,-1,-1,0,csAlpha, 'uvw'))
```

Symmetrically Equivalent Misorientations

- ▶ sA - symmetry operation from csAlpha
- ▶ sB - symmetry operation from csBeta
- ▶ beta2alpha - misorientation from beta to alpha
- ▶ symmetrically equivalent misorientations:
sA * beta2alpha * sB
- ▶ number of symmetric misorientations:
 - ▶ 2 Laue groups: $|csAlpha| * |csBeta| / 2$
 - ▶ otherwise: $|csAlpha| * |csBeta|$

beta2alpha . **symmetrise**

```
ans = misorientation (Ti \(beta\) -> Ti \(alpha\))  
size: 288 x 1  
  
show Euler angles
```

Grain Exchange Symmetry

- ▶ consider misorientations $\text{inv}(\text{ori1}) * \text{ori2}$ between orientations of the same phase
- ▶ sometimes no ordering $\text{ori1}, \text{ori2}$ is given e.g. boundary misorientations
- ▶ $\rightarrow$ we should not distinguish between $\text{inv}(\text{ori1}) * \text{ori2}$ vs $\text{inv}(\text{ori2}) * \text{ori1}$
- ▶ this additional symmetry is called grain exchange symmetry
- ▶ in MTEX it is denoted by the option 'antipodal'

```
mori = grains.boundary.misorientation  
mori.antipodal
```

```
ans = misorientation (Ti (beta) -> Ti (beta))  
size: 8931 x 1  
antipodal: true
```

Misorientation Angle and Axis

misorientation

```
mori = inv(ori1)*ori2
```

the **misorientation angle** is the smallest rotational angle of **mori.symmetrise**

```
angle(ori1 , ori2 )  
angle(mori)
```

the **disorientation** is the misorientation with

- ▶ minimum angle
- ▶ axis in the fundamental sector

```
mori.project2FundamentalRegion
```

the rotational angle ignoring symmetry

```
angle(mori , 'noSymmetry')
```

the maximum rotational angle

```
angle(mori , 'max')
```

the rotational axis of **mori** refers to the disorientation

```
axis(mori , csAlpha )  
axis(mori , csBeta )
```

the axis in specimen coordinates

```
axis(ori1 , ori2 )
```

ignore symmetry

```
axis(mori , 'noSymmetry')
```

the axis for the maximum rotational angle

```
axis(mori , 'max')
```

Misorientation Angle and Axis

misorientation

```
mori = inv(ori1)*ori2
```

the **misorientation angle** is the smallest rotational angle of **mori.symmetrise**

```
angle(ori1 , ori2)  
angle(mori)
```

the **disorientation** is the misorientation with

- ▶ minimum angle
- ▶ axis in the fundamental sector

```
mori.project2FundamentalRegion
```

the rotational angle ignoring symmetry

```
angle(mori , 'noSymmetry')
```

the maximum rotational angle

```
angle(mori , 'max')
```

the rotational axis of **mori** refers to the disorientation

```
axis(mori , csAlpha)  
axis(mori , csBeta)
```

the axis in specimen coordinates

```
axis(ori1 , ori2)
```

ignore symmetry

```
axis(mori , 'noSymmetry')
```

the axis for the maximum rotational angle

```
axis(mori , 'max')
```

Misorientation Angle and Axis

misorientation

```
mori = inv(ori1)*ori2
```

the **misorientation angle** is the smallest rotational angle of **mori.symmetrise**

```
angle(ori1 , ori2)  
angle(mori)
```

the **disorientation** is the misorientation with

- ▶ minimum angle
- ▶ axis in the fundamental sector

```
mori.project2FundamentalRegion
```

the rotational angle ignoring symmetry

```
angle(mori , 'noSymmetry')
```

the maximum rotational angle

```
angle(mori , 'max')
```

the rotational axis of **mori** refers to the disorientation

```
axis(mori , csAlpha)  
axis(mori , csBeta)
```

the axis in specimen coordinates

```
axis(ori1 , ori2)
```

ignore symmetry

```
axis(mori , 'noSymmetry')
```

the axis for the maximum rotational angle

```
axis(mori , 'max')
```


Misorientation Angle and Axis

misorientation

```
mori = inv(ori1)*ori2
```

the **misorientation angle** is the smallest rotational angle of **mori.symmetrise**

```
angle(ori1 , ori2)  
angle(mori)
```

the **disorientation** is the misorientation with

- ▶ minimum angle
- ▶ axis in the fundamental sector

```
mori.project2FundamentalRegion
```

the rotational angle ignoring symmetry

```
angle(mori , 'noSymmetry')
```

the maximum rotational angle

```
angle(mori , 'max')
```

the rotational axis of **mori** refers to the disorientation

```
axis(mori , csAlpha)  
axis(mori , csBeta)
```

the axis in specimen coordinates

```
axis(ori1 , ori2)
```

ignore symmetry

```
axis(mori , 'noSymmetry')
```

the axis for the maximum rotational angle

```
axis(mori , 'max')
```

Misorientation Angle and Axis

misorientation

```
mori = inv(ori1)*ori2
```

the **misorientation angle** is the smallest rotational angle of **mori.symmetrise**

```
angle(ori1 , ori2)  
angle(mori)
```

the **disorientation** is the misorientation with

- ▶ minimum angle
- ▶ axis in the fundamental sector

```
mori.project2FundamentalRegion
```

the rotational angle ignoring symmetry

```
angle(mori , 'noSymmetry')
```

the maximum rotational angle

```
angle(mori , 'max')
```

the rotational axis of **mori** refers to the disorientation

```
axis(mori , csAlpha)  
axis(mori , csBeta)
```

```
ans = Miller (Ti (alpha))  
      h      k      i      l  
-1.5812  2.9291 -1.348  0.5158  
  
ans = Miller (Ti (beta))  
      h      k      l  
-0.1511  3.2874  0.3649
```

the axis in specimen coordinates

```
axis(ori1 , ori2)
```

ignore symmetry

```
axis(mori , 'noSymmetry')
```

the axis for the maximum rotational angle

```
axis(mori , 'max')
```

Misorientation Angle and Axis

misorientation

```
mori = inv(ori1)*ori2
```

the **misorientation angle** is the smallest rotational angle of **mori.symmetrise**

```
angle(ori1 , ori2)  
angle(mori)
```

the **disorientation** is the misorientation with

- ▶ minimum angle
- ▶ axis in the fundamental sector

```
mori . project2FundamentalRegion
```

the rotational angle ignoring symmetry

```
angle(mori , 'noSymmetry')
```

the maximum rotational angle

```
angle(mori , 'max')
```

the rotational axis of **mori** refers to the disorientation

```
axis(mori , csAlpha)  
axis(mori , csBeta)
```

the axis in specimen coordinates

```
axis(ori1 , ori2)
```

```
ans = vector3d (show methods , plot)  
           x           y           z  
0.0495585  0.992437  -0.112305
```

ignore symmetry

```
axis(mori , 'noSymmetry')
```

the axis for the maximum rotational angle

```
axis(mori , 'max')
```

Misorientation Angle and Axis

misorientation

```
mori = inv(ori1)*ori2
```

the **misorientation angle** is the smallest rotational angle of **mori.symmetrise**

```
angle(ori1 , ori2)  
angle(mori)
```

the **disorientation** is the misorientation with

- ▶ minimum angle
- ▶ axis in the fundamental sector

```
mori . project2FundamentalRegion
```

the rotational angle ignoring symmetry

```
angle(mori , 'noSymmetry')
```

the maximum rotational angle

```
angle(mori , 'max')
```

the rotational axis of **mori** refers to the disorientation

```
axis(mori , csAlpha)  
axis(mori , csBeta)
```

the axis in specimen coordinates

```
axis(ori1 , ori2)
```

ignore symmetry

```
axis(mori , 'noSymmetry')
```

the axis for the maximum rotational angle

```
axis(mori , 'max')
```

Misorientation Angle and Axis

misorientation

```
mori = inv(ori1)*ori2
```

the **misorientation angle** is the smallest rotational angle of **mori.symmetrise**

```
angle(ori1 , ori2)  
angle(mori)
```

the **disorientation** is the misorientation with

- ▶ minimum angle
- ▶ axis in the fundamental sector

```
mori . project2FundamentalRegion
```

the rotational angle ignoring symmetry

```
angle(mori , 'noSymmetry')
```

the maximum rotational angle

```
angle(mori , 'max')
```

the rotational axis of **mori** refers to the disorientation

```
axis(mori , csAlpha)  
axis(mori , csBeta)
```

the axis in specimen coordinates

```
axis(ori1 , ori2)
```

ignore symmetry

```
axis(mori , 'noSymmetry')
```

the axis for the maximum rotational angle

```
axis(mori , 'max')
```

Misorientation Angle and Axis

misorientation

```
mori = inv(ori1)*ori2
```

the **misorientation angle** is the smallest rotational angle of **mori.symmetrise**

```
angle(ori1 , ori2)  
angle(mori)
```

the **disorientation** is the misorientation with

- ▶ minimum angle
- ▶ axis in the fundamental sector

```
mori . project2FundamentalRegion
```

the rotational angle ignoring symmetry

```
angle(mori , 'noSymmetry')
```

the maximum rotational angle

```
angle(mori , 'max')
```

the rotational axis of **mori** refers to the disorientation

```
axis(mori , csAlpha)  
axis(mori , csBeta)
```

the axis in specimen coordinates

```
axis(ori1 , ori2)
```

ignore symmetry

```
axis(mori , 'noSymmetry')
```

the axis for the maximum rotational angle

```
axis(mori , 'max')
```

Axis Angle Coloring

the standard color key

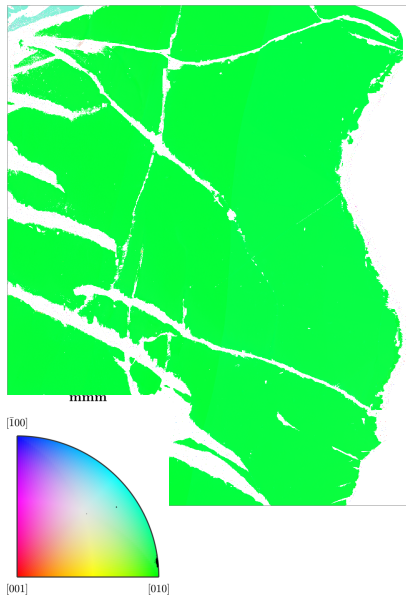
```
ori = ebsd( 'Fo' ).orientations  
key = ipfHSVKey(ori.CS)  
color = key.orientation2color(ori)
```

sharp standard color key

```
key.inversePoleFigureDirection =  
    mean(ori)* key.whiteCenter  
key.maxAngle = 4*degree
```

sharp axis angle key

```
key = axisAngleKey  
key.oriRef = mean(ori)  
key.maxAngle = 4*degree
```



Axis Angle Coloring

the standard color key

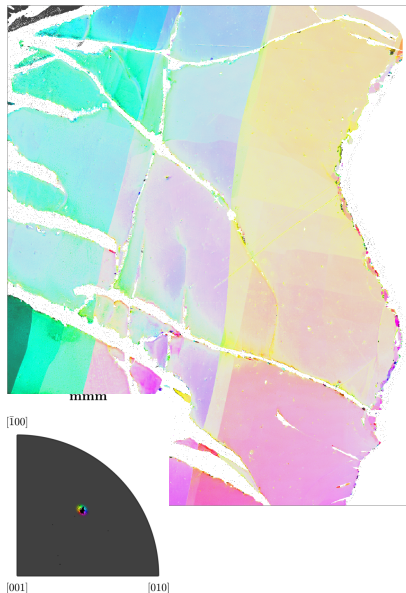
```
ori = ebsd( 'Fo' ).orientations  
key = ipfHSVKey(ori.CS)  
color = key.orientation2color(ori)
```

sharp standard color key

```
key.inversePoleFigureDirection =  
    mean(ori)* key.whiteCenter  
key.maxAngle = 4*degree
```

sharp axis angle key

```
key = axisAngleKey  
key.oriRef = mean(ori)  
key.maxAngle = 4*degree
```



Axis Angle Coloring

the standard color key

```
ori = ebsd( 'Fo' ).orientations  
key = ipfHSVKey(ori.CS)  
color = key.orientation2color(ori)
```

sharp standard color key

```
key.inversePoleFigureDirection =  
    mean(ori)* key.whiteCenter  
key.maxAngle = 4*degree
```

sharp axis angle key

```
key = axisAngleKey  
key.oriRef = mean(ori)  
key.maxAngle = 4*degree
```



Grain Reference Orientation Deviation and mis2mean

- ▶ `ori` - orientation at a certain pixel
- ▶ `oriG_g` - grain mean orientation
- ▶ `S` - symmetry to minimize the angle of

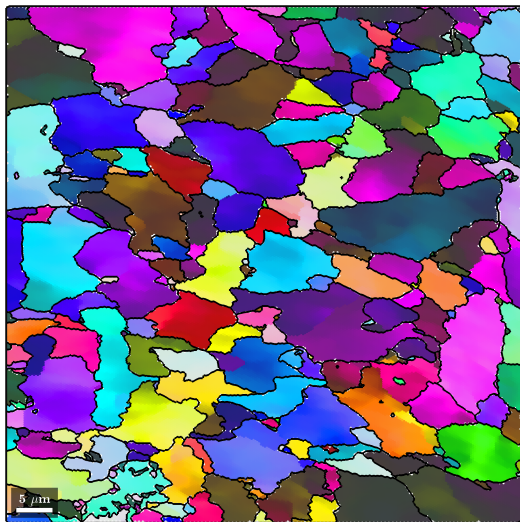
```
GROD = S * inv(ori_g) * ori
```

```
grod = calcGROD(ebsd, grains)
```

```
GOS = grainMean(ebsd, grod.angle)
```

```
axC = grod.axis
```

```
axS = ebsd.orientations .* ...  
      grod.axis('noSymmetry')
```



Grain Reference Orientation Deviation and mis2mean

- ▶ `ori` - orientation at a certain pixel
- ▶ `oriG_g` - grain mean orientation
- ▶ `S` - symmetry to minimize the angle of

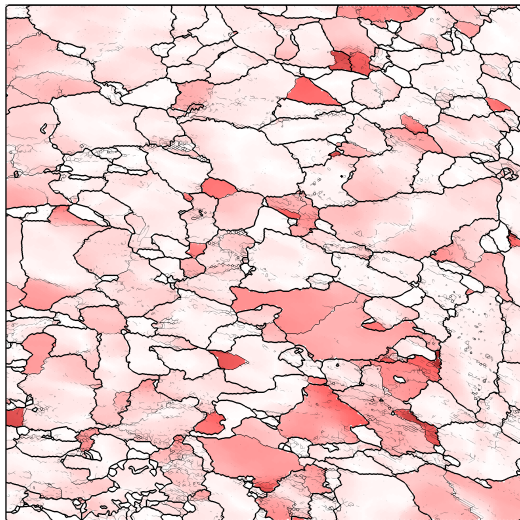
```
GROD = S * inv(ori_g) * ori
```

```
grod = calcGROD(ebsd, grains)
```

```
GOS = grainMean(ebsd, grod.angle)
```

```
axC = grod.axis
```

```
axS = ebsd.orientations .* ...  
      grod.axis('noSymmetry')
```



Grain Reference Orientation Deviation and mis2mean

- ▶ `ori` - orientation at a certain pixel
- ▶ `oriG_g` - grain mean orientation
- ▶ `S` - symmetry to minimize the angle of

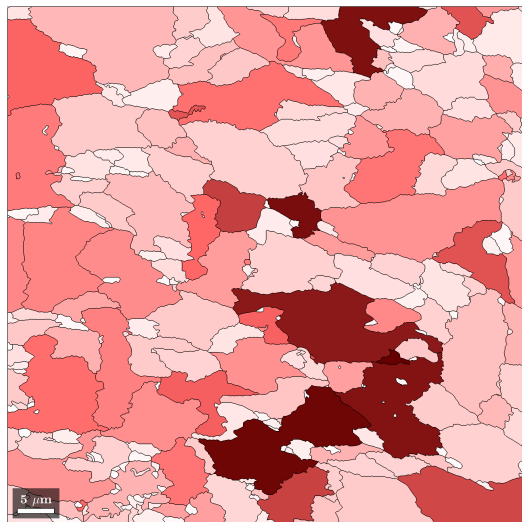
```
GROD = S * inv(ori_g) * ori
```

```
grod = calcGROD(ebsd, grains)
```

```
GOS = grainMean(ebsd, grod.angle)
```

```
axC = grod.axis
```

```
axS = ebsd.orientations .* ...  
      grod.axis('noSymmetry')
```



Grain Reference Orientation Deviation and mis2mean

- ▶ `ori` - orientation at a certain pixel
- ▶ `oriG_g` - grain mean orientation
- ▶ `S` - symmetry to minimize the angle of

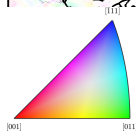
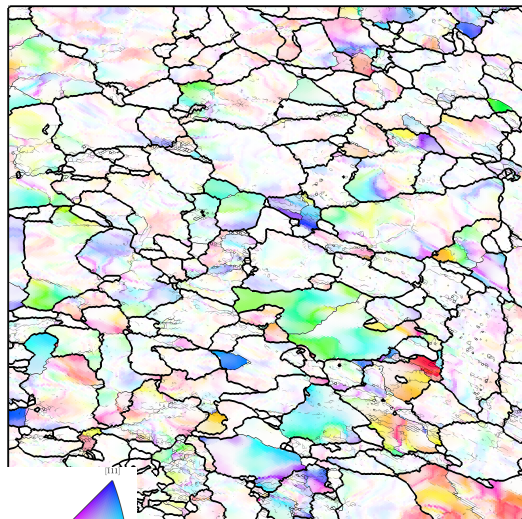
```
GROD = S * inv(ori_g) * ori
```

```
grod = calcGROD(ebsd, grains)
```

```
GOS = grainMean(ebsd, grod.angle)
```

```
axC = grod.axis
```

```
axS = ebsd.orientations .* ...  
      grod.axis('noSymmetry')
```



Grain Reference Orientation Deviation and mis2mean

- ▶ `ori` - orientation at a certain pixel
- ▶ `oriG_g` - grain mean orientation
- ▶ `S` - symmetry to minimize the angle of

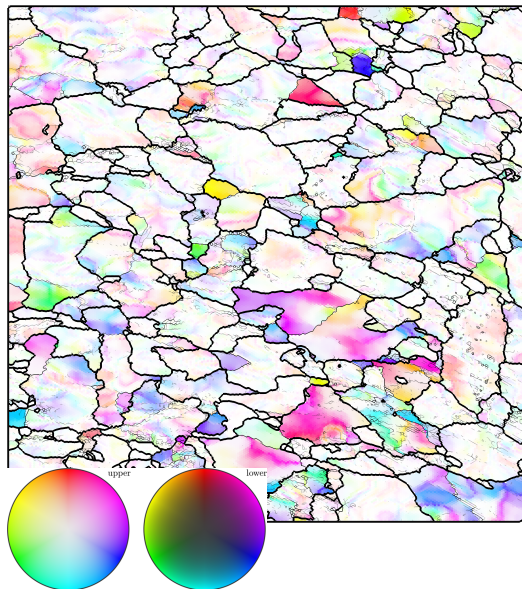
```
GROD = S * inv(ori_g) * ori
```

```
grod = calcGROD(ebsd, grains)
```

```
GOS = grainMean(ebsd, grod.angle)
```

```
axC = grod.axis
```

```
axS = ebsd.orientations .* ...  
      grod.axis('noSymmetry')
```



Misorientation Distribution

distribution of boundary misorientation angles

```
mori = grains.boundary.misorientation  
histogram(mori.angle./degree)  
plotAngleDistribution(mori)
```

uniform misorientation distribution function

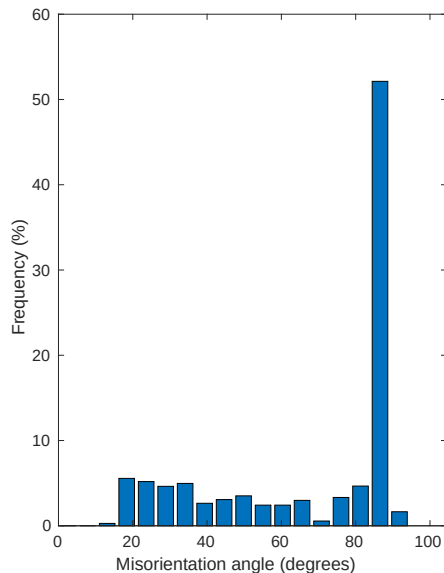
```
plotAngleDistribution(mori.CS)
```

uncorrelated misorientation distribution function

```
odf = calcODF(ebbsd.orientations)  
umdf = calcMDF(odf)  
plotAngleDistribution(umdf)
```

distribution of boundary misorientations

```
bmddf = calcDensity(mori)  
plotAngleDistribution(bmddf)
```



Misorientation Distribution

distribution of boundary misorientation angles

```
mori = grains.boundary.misorientation  
histogram(mori.angle./degree)  
plotAngleDistribution(mori)
```

uniform misorientation distribution function

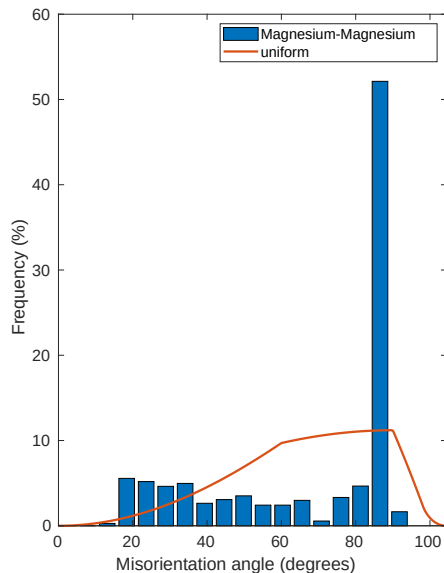
```
plotAngleDistribution(mori.CS)
```

uncorrelated misorientation distribution function

```
odf = calcODF(ebbsd.orientations)  
umdf = calcMDF(odf)  
plotAngleDistribution(umdf)
```

distribution of boundary misorientations

```
bmfd = calcDensity(mori)  
plotAngleDistribution(bmfd)
```



Misorientation Distribution

distribution of boundary misorientation angles

```
mori = grains.boundary.misorientation  
histogram(mori.angle./degree)  
plotAngleDistribution(mori)
```

uniform misorientation distribution function

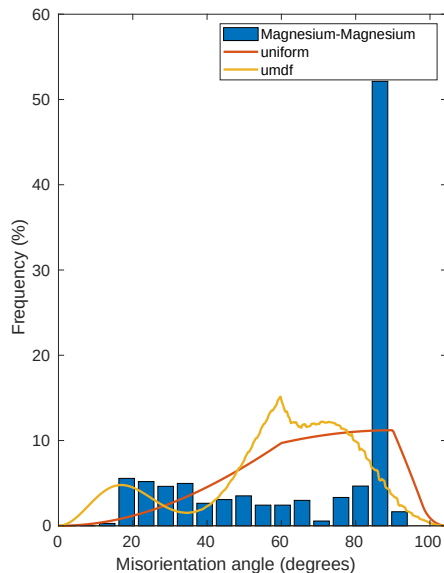
```
plotAngleDistribution(mori.CS)
```

uncorrelated misorientation distribution function

```
odf = calcODF(ebsd.orientations)  
umdf = calcMDF(odf)  
plotAngleDistribution(umdf)
```

distribution of boundary misorientations

```
bmfd = calcDensity(mori)  
plotAngleDistribution(bmfd)
```



Misorientation Distribution

distribution of boundary misorientation angles

```
mori = grains.boundary.misorientation  
histogram(mori.angle./degree)  
plotAngleDistribution(mori)
```

uniform misorientation distribution function

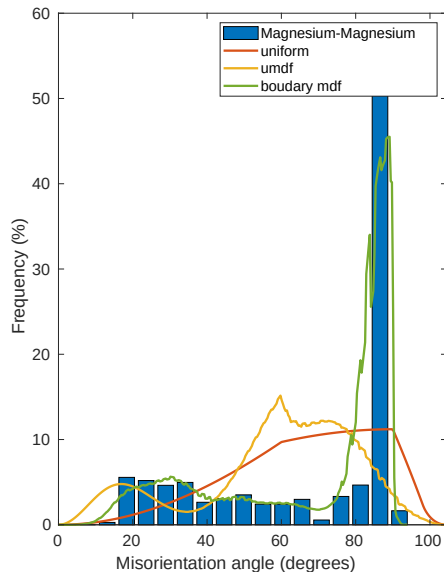
```
plotAngleDistribution(mori.CS)
```

uncorrelated misorientation distribution function

```
odf = calcODF(ebbsd.orientations)  
umdf = calcMDF(odf)  
plotAngleDistribution(umdf)
```

distribution of boundary misorientations

```
bmfd = calcDensity(mori)  
plotAngleDistribution(bmfd)
```



Misorientation Axis Distribution

distribution of boundary misorientation angles

```
plotAxisDistribution(mori)
```

```
plotAxisDistribution(mori, 'contourf')
```

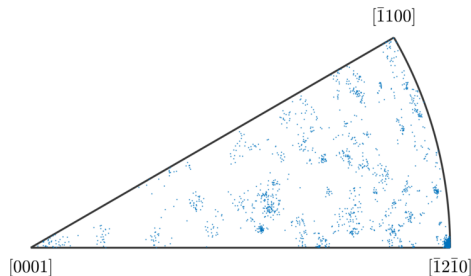
uniform misorientation distribution function

```
plotAxisDistribution(mori.CS)
```

uncorrelated misorientation distribution function

texture dependent

```
plotAxisDistribution(umdf)
```



Misorientation Axis Distribution

distribution of boundary misorientation angles

```
plotAxisDistribution(mori)
```

```
plotAxisDistribution(mori, 'contourf')
```

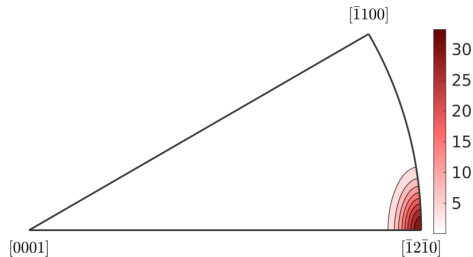
uniform misorientation distribution function

```
plotAxisDistribution(mori.CS)
```

uncorrelated misorientation distribution function

texture dependent

```
plotAxisDistribution(umdf)
```



Misorientation Axis Distribution

distribution of boundary misorientation angles

```
plotAxisDistribution(mori)
```

```
plotAxisDistribution(mori, 'contourf')
```

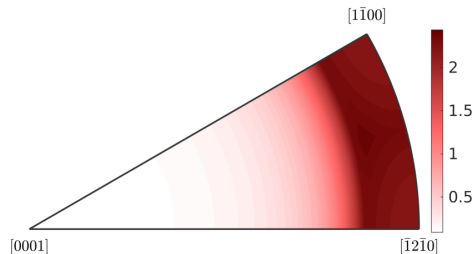
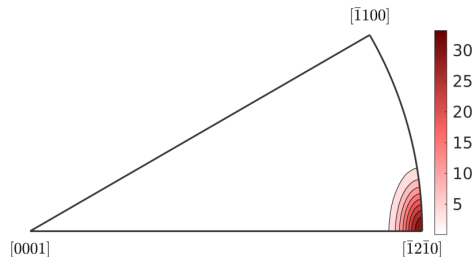
uniform misorientation distribution function

```
plotAxisDistribution(mori.CS)
```

uncorrelated misorientation distribution function

texture dependent

```
plotAxisDistribution(umdf)
```



Misorientation Axis Distribution

distribution of boundary misorientation angles

```
plotAxisDistribution(mori)
```

```
plotAxisDistribution(mori, 'contourf')
```

uniform misorientation distribution function

```
plotAxisDistribution(mori.CS)
```

uncorrelated misorientation distribution function

texture dependent

```
plotAxisDistribution(umdf)
```

